Project Title: Advancing Autonomous Vehicle Safety Through Occupancy & Flow Prediction

Name: Diksha Menghmalani, 1st-year Master's student
Department of Computer Science,
Contact: `menghmalani@wisc.edu`

Name: Gokulnath Sourirajan, 1st-year Master's student
Department of Computer Science,
Contact: `sourirajan@wisc.edu`

Name: Thilak Raj Murugan, 1st-year Master's student
Department of Computer Science,
Contact: `tmurugan2@wisc.edu`
[1]

Course Number: STAT 453, Spring 2025
Team number: 24

File compiled on Overleaf

[1] Names added in alphabetical order

1 Introduction

1.1 Motivation.

Autonomous vehicles (AVs) have the potential to drastically reduce traffic accidents, yet their widespread adoption depends on solving critical safety challenges. According to global statistics, approximately 1.2 million people die annually in car crashes, with millions more injured. A major contributing factor is the inability of current AV systems to accurately predict the movements of other road agents—especially in complex, real-world scenarios involving occlusions, erratic driver behavior, and multi-vehicle interactions. Traditional path-planning algorithms rely heavily on static rules and short-term predictions, leaving AVs vulnerable to sudden obstacles or hidden pedestrians. To address this, our project focuses on occupancy and flow prediction, a dynamic modeling approach that enables AVs to anticipate future road conditions up to 8 seconds in advance. By improving long-term forecasting and accounting for occluded objects, this work aims to enhance collision avoidance, optimize navigation efficiency, and ultimately save lives.

1.2 Existing literature.

Recent advancements in deep learning have explored various methods for traffic prediction, yet significant limitations remain. One prominent approach, "Learning Traffic as Images" [3], treats traffic data as 2D spatiotemporal images and applies convolutional neural networks (CNNs) to detect congestion patterns. While effective for highway speed prediction, this method fails to model occluded vehicles or pedestrians and assumes perfect compliance with traffic rules—an unrealistic assumption in real-world driving. Another study, "Spatio-Temporal Vehicle Traffic Flow Prediction," [4] combines CNNs with long short-term memory (LSTM) networks to improve short-term (1–3 second) forecasts, particularly in stop-and-go traffic. However, this hybrid model still struggles with longer prediction horizons, non-compliant road users, and occluded objects. Crucially, both approaches treat vehicles as independent agents, ignoring the complex interactions that define real traffic scenarios. These gaps highlight the need for a more holistic prediction framework that accounts for multi-agent dynamics, occlusions, and unpredictable behaviors.

1.3 Limitations of existing models.

Despite their contributions, current traffic prediction models suffer from several key shortcomings. First, their **prediction horizons are too short** (typically 1–3 seconds), limiting their usefulness for proactive AV decision-making. Second, they **do not account for occluded objects**, a critical flaw in urban environments where pedestrians or cars may suddenly emerge from blind spots. Third, most models **assume idealized traffic conditions**, ignoring common violations such as jaywalking, illegal lane changes, or aggressive driving. Finally, existing approaches **fail to model multi-agent interactions**, treating each vehicle's trajectory as independent rather than interdependent. For example, when two cars approach an intersection simultaneously, their movements are influenced by each other—a factor overlooked by per-vehicle prediction methods. These limitations result in brittle systems that perform well in simulations but struggle in real-world chaos.

1.4 Task

The objective of our model is to predict vehicle occupancy (both observed and occluded) and flow vectors (motion direction and magnitude) over a 2D bird's-eye view (BEV) grid representing the road scene, particularly:

- **Observed occupancy**: Future positions of visible vehicles over an 8-second horizon (1-second intervals),
- **Occluded occupancy**: Likely locations of hidden agents (e.g., pedestrians behind obstacles), and
- **Flow**: Motion vectors (direction and magnitude) for all agents.

Occupancy prediction is treated as a classification problem, where each cell in the grid is classified as occupied or not. Flow prediction is a regression task, where the model outputs the expected displacement vectors (dx, dy) for each grid cell.

2 Datasets

The Waymo Open Motion Dataset (WOMD), a large-scale autonomous driving dataset comprises 574 hours of motion data spanning approximately 1,750 kilometers of roadways, with a total data size of around 1 TB. The data was collected across six major U.S. cities—San Francisco, Phoenix, Mountain View, Los Angeles, Detroit, and Seattle, offering diverse urban driving conditions. The dataset provides rich spatiotemporal information through LIDAR and multi-view camera embeddings, including inputs from front, front-left, front-right, side-left, side-right, rear-left, and rear-right sensors. This dataset plays a crucial role in traffic flow prediction, which is essential for driver assistance systems and AV decision-making.

For this project, we utilized a sampled subset of 33 GB, where each input sample consists of 91 consecutive frames capturing dynamic road scenes. Each frame includes detailed annotations of agent positions, velocities, and movement directions (flow) for various road users such as vehicles, pedestrians, and cyclists, making it highly suitable for training models focused on occupancy and flow prediction. Leveraging the *Waymo Open Dataset*, our model ingests a 1 second historical context of agent trajectories and scene semantics to generate these predictions.

3 Model Architecture

Our model follows an encoder-decoder architecture. The model contains approximately 37 million trainable parameters and is optimized using a multi-task loss that combines classification and regression objectives. The input is a 256*256*23 tensor with 23 channels and the output is a 256*256*32 tensor.

3.1 Encoder

A modified ResNet-50V2 (pretrained on ImageNet and stripped of its top classification layer) is used to extract high-level spatial features from the input. The encoder processes the multi-sensor temporal embeddings and compresses the input into a lower-dimensional latent representation.

3.2 Decoder

The decoder consists of a 5 blocks of upsampling pipeline, where each block includes:

- A `Conv2D` layer to refine features,
- An `Upsampling` operation to increase spatial resolution,
- Another `Conv2D` layer.

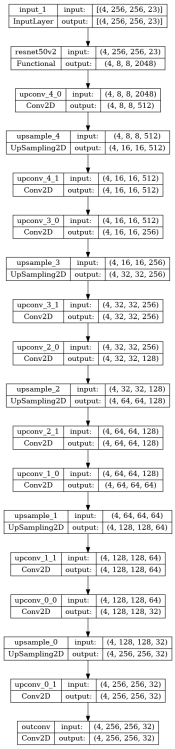


Figure 1: Model Architecture

3.3 Preprocessing Input

To prepare the data for model training, we transform raw position and trajectory data into structured spatial-temporal representations suitable for learning.

Each scene is encoded into a set of **timestep grids**, generated separately for each agent type at a given time stamp. Agents can be vehicles, pedestrians, and cyclists. These grids capture:

- **Occupancy**: Indicates whether a cell in the timegrid is occupied.
  - **Observed**: Marks cells where the agents are directly visible.
  - **Occluded**: Denotes cells where an agent is hidden or outside the sensor view.
- **Flow**: Stores motion vectors (dx, dy) for occupied cells.

In addition, a static **road graph** layer is included to provide contextual road layout information (e.g., lane boundaries, intersections). We construct a multi-channel input tensor that incorporates both the **past 1 second (10 frames/sec)** and **current** occupancy information:

- 1 channel for the road graph.
- 10 channels for vehicle occupancy in the past and 1 for current = 11 channels.
- 10 channels for merged pedestrian and cyclist occupancy in the past and 1 for current = 11 channels.

This results in a total of 23 channels per sample. The final input tensor shape is $[n, 256, 256, 23]$, where n denotes the batch size.

3.4 Output

For each timestep, the output is a 4 channel 2D grid representing future occupancy and motion:

- 1 channel for observed occupancy (classification),
- 1 channel for occluded occupancy (classification),
- 2 channels for flow vectors (`dx`, `dy`, regression).

Since we are predicting the next 8 timesteps, the model's output layer consists of 32 channels.

4 Loss Function and Hyperparameters

To train the model on both classification and regression objectives, we use a composite loss function defined as:

$$\begin{aligned}\text{Total Loss} &= \mathcal{L}^{\text{CE}}_{\text{observed}} + \mathcal{L}^{\text{CE}}_{\text{occluded}} + \mathcal{L}^{\text{L2}}_{\text{flow}} \\ &= \text{CE(Observed Occupancy)} + \text{CE(Occluded Occupancy)} + \text{L2 Error(Flow)}\end{aligned}$$

The cross entropy loss is denoted by CE. This formulation ensures the model learns accurate occupancy classification while also predicting fine-grained motion vectors through L2 regression.

We experimented with a range of hyperparameters. Learning rates tested include 0.001, 0.005, 0.01, and 0.05. Both Adam and SGD optimizers were evaluated during training. Batch sizes of 16, 32, and 64 were explored to balance convergence speed and memory usage. Final choices were selected based on validation set performance.

5 Metrics

To evaluate the performance of our model in predicting both occupancy and flow, we use multiple metrics. Each metric captures a different aspect of prediction quality. For example, some focus on how accurate the predictions are in identifying occupied regions, while others focus on how closely predicted vehicle movements match the actual ones. This variety ensures we're not optimizing for just one type of error while missing others.

5.1 Metrics for Occupancy

Occupancy prediction is treated as a classification problem — for each cell in a spatial grid, the model decides whether it will be occupied in the future. To evaluate this, we use two main metrics: AUC and Soft-IoU. Let:

- O^b_k: Ground-truth occupancy at future waypoint k for currently-observed vehicles.
- $\hat{O}^b_k$: Predicted occupancy at waypoint k for currently-observed vehicles.
- O^c_k, $\hat{O}^c_k$: Ground-truth and predicted occupancy for currently-occluded vehicles.

AUC (Area Under the Curve):

We use AUC to summarize the trade-off between precision and recall at different decision thresholds. The AUC gives us a sense of how good the model is at distinguishing between occupied and empty cells.

For each waypoint k, we compute:

$$\text{AUC}(O^b_k, \hat{O}^b_k), \quad \text{AUC}(O^c_k, \hat{O}^c_k)$$

for currently-observed and currently-occluded vehicles, respectively.

Soft-IoU (Soft Intersection over Union):

IoU measures the overlap between predicted and true occupied regions. But because our model gives soft values (like probabilities) rather than hard yes/no answers, we use Soft-IoU. This version considers partial correctness — a prediction close to 1 in an actually occupied cell is better than one near 0.

Soft-IoU for currently-observed vehicles is computed as:

$$\text{Soft-IoU}(O_k^b, \hat{O}_k^b) = \frac{\sum_{x,y} O_k^b \hat{O}_k^b}{\sum_{x,y} O_k^b + \hat{O}_k^b - O_k^b \hat{O}_k^b}$$

where x, y index the spatial coordinates in the grid. If the ground truth O_k^b is empty (no occupied cells), the metric is set to zero. Final scores are averaged over all 8 future waypoints. A similar expression is used for occluded vehicles:

$$\text{Soft-IoU}(O_k^c, \hat{O}_k^c)$$

5.2 Metrics for Flow

Flow prediction is a regression task — we want the model to estimate how vehicles will move over time. Here, we evaluate how close the predicted motion vectors are to the ground truth using the End-Point Error. Let:

- $F_k(x, y)$: Ground-truth flow vector at spatial location (x, y) for future waypoint k.
- $\hat{F}_k(x, y)$: Predicted flow vector at the same location.

End-Point Error (EPE):

EPE measures the average distance between the predicted and actual motion of each grid cell. Intuitively, it's like drawing arrows for actual movement and predicted movement, then measuring how far apart their tips are. The smaller this value, the more accurate the motion prediction.

EPE is defined as:

$$\left\| F_k(x, y) - \hat{F}_k(x, y) \right\|_2 \quad \text{where} \quad F_k(x, y) \neq (0, 0)$$

We compute the L2 distance only for locations where actual motion exists (ie. if the cell grid was occupied) and average it across all future waypoints.

6 Results

We trained our model for a maximum of 40 epochs. To avoid overfitting and ensure the best possible generalization, we employed an early stopping strategy. This allowed us to automatically identify and retain the model weights from the epoch where validation performance peaked. All reported results and visualizations are based on this best-performing checkpoint.

We evaluate our model using the metrics defined previously for both occupancy and flow prediction tasks. All values are averaged over 8 predicted future waypoints. Multiple metrics are reported to capture different aspects of prediction performance. AUC and Soft-IoU assess occupancy prediction accuracy, while End-Point Error (EPE) quantifies how close our predicted flow vectors are to the true vehicle movements.

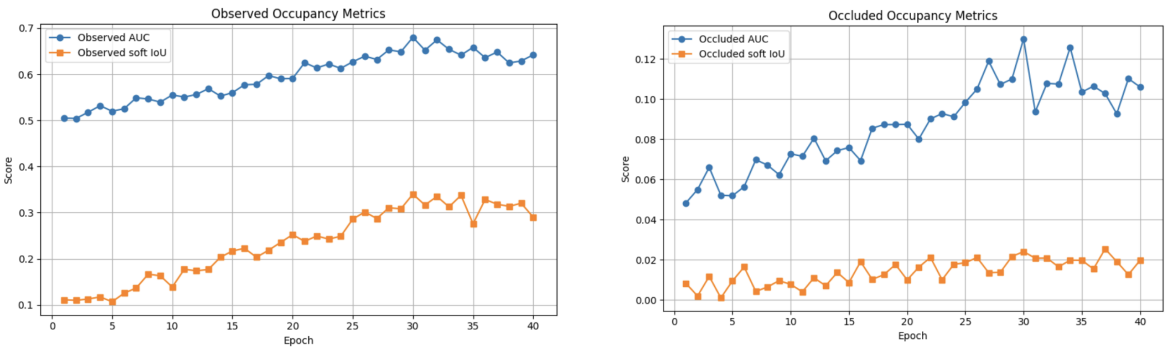
Occupancy Results. Our model achieves an AUC of 0.68 and a Soft-IoU of 0.34 for currently-observed vehicles, indicating a moderate ability to identify occupied grid cells accurately. For currently-occluded vehicles, however, the scores drop significantly—AUC falls

Task	Metric	Value
Observed Occupancy	AUC	0.68
Observed Occupancy	Soft-IoU	0.34
Occluded Occupancy	AUC	0.13
Occluded Occupancy	Soft-IoU	0.024
Flow Prediction	EPE (End-Point Error)	4.2

Table 1: Model performance averaged over 8 future waypoints.

to 0.13 and Soft-IoU to 0.024. While this suggests the model struggles under occlusion, it is also reflective of the intrinsic difficulty of the task: predicting what is not directly visible is fundamentally harder and often ambiguous.

Importantly, lower scores on occluded occupancy prediction do not necessarily mean the model is ineffective—they highlight the nature of the challenge. Even if the model cannot precisely predict occluded regions, it can still be made aware of uncertainty due to impaired vision. Moving forward, we plan to improve the model's robustness by explicitly incorporating **occlusion-awareness**, for example through attention mechanisms that weigh visible context more heavily, or by learning to predict areas of high uncertainty. Such strategies could help the model make safer, more informed decisions under partial observability.



(a) Observed Occupancy Metrics across Epochs (b) Occluded Occupancy Metrics across Epochs

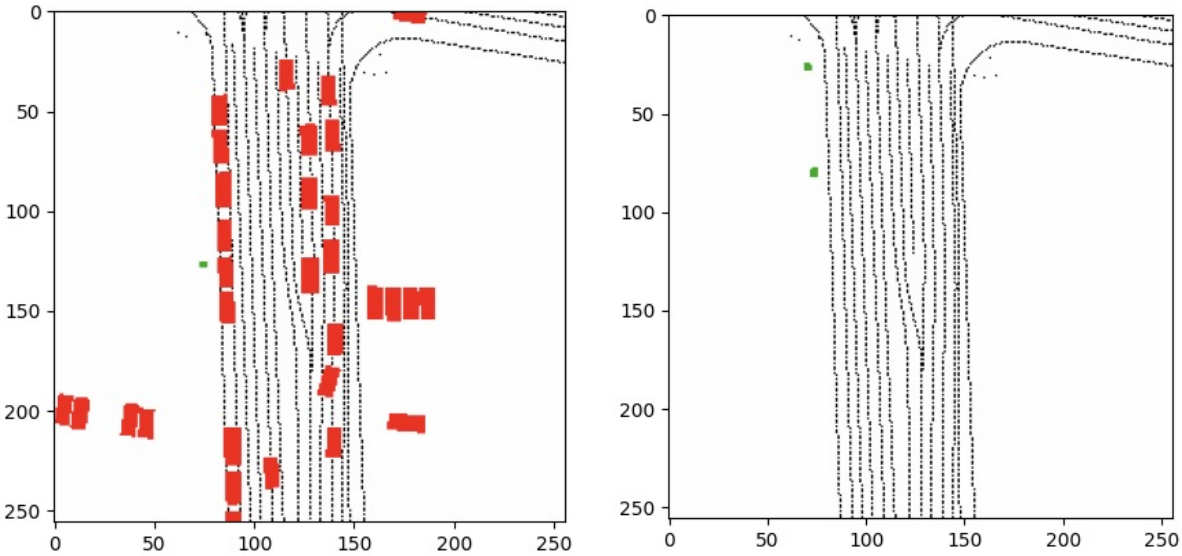
Figure 2: Training curves for occupancy metrics. The model quickly learns to predict observed occupancy but struggles with occluded regions due to missing visual input.

Flow Results. For flow prediction, our model achieves an End-Point Error (EPE) of 4.2. This represents the average L2 distance between the predicted and ground-truth flow vectors. While the result is reasonable for scenes involving complex multi-agent motion, there remains room for improvement in trajectory precision.

The training curve and a representative output are shown below.

Key Findings.

- **Transfer learning helped kickstart training.**
  - We initialized our model with weights pre-trained on a related task, which significantly reduced the number of epochs needed to converge.
  - This was especially useful given the limited size of our dataset; training from scratch led to slower and more unstable learning.



(a) Observed Occupancy Prediction Example

(b) Occluded Occupancy Prediction Example

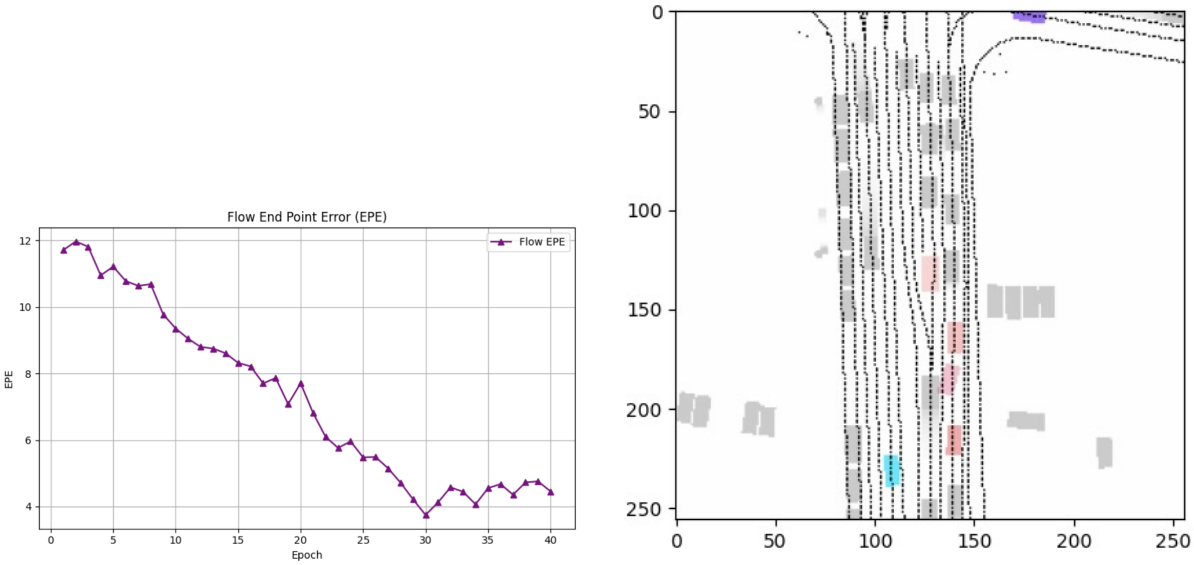
Figure 3: Sample predictions from the model. Observed occupancy is captured well, while occluded regions remain sparse.

  - Transfer learning allowed the model to generalize early on, focusing training on scene-specific fine-tuning rather than learning basic representations from zero.

- **We used random search to establish a good starting point for tuning.**
  - We randomly sampled hyperparameters like learning rate, dropout, and latent dimensions to find combinations that gave reasonable train/validation loss.
  - Rather than exhaustively trying every combination upfront, this gave us faster feedback and avoided early overfitting or underfitting.
  - Once a promising setup was found, we refined key parameters manually based on how the training curves evolved over time.

- **Hyperparameter tuning had a major impact on overall performance.**
  - Even small changes to latent size or dropout significantly influenced both occupancy and flow prediction quality.
  - We adjusted parameters based on validation performance, and often re-ran training to isolate the effect of one variable at a time.
  - Overfitting was controlled by early stopping and weight decay, especially as the model grew more confident on the observed occupancy task.
  - The best-performing hyperparameter configuration was:
    * Batch size: **32**
    * Optimizer: **Adam**
    * Learning rate: **0.005**
  - This configuration provided a good balance between convergence speed and generalization across all three output types (observed occupancy, occluded occupancy, and flow).

- **Adam outperformed other optimizers by stabilizing learning.**



(a) EPE across Training Epochs

(b) Sample Flow Prediction Output

Figure 4: Flow prediction performance over time and sample output visualization.

  - Other optimizers like SGD and RMSProp either failed to converge or were too sensitive to initial learning rates.
  - Adam's adaptive learning rate was crucial in avoiding sharp loss oscillations, especially in the early training stages.
  - It helped balance updates across layers, which is particularly helpful when working with multiple prediction heads (occupancy and flow).

- **The model still struggles with occlusions, but this is expected.**
  - The performance drop for occluded occupancy is not solely a model failure—it reflects the ambiguity of predicting objects outside the field of view.
  - Despite this, the model can still learn to *anticipate occlusion scenarios*, such as predicting uncertainty in blind spots.
  - Future work can focus on improving occlusion-awareness by using attention mechanisms, sensor fusion, or uncertainty modeling to signal impaired visibility.

7 Computation Cost

Cluster Resources. All experiments were executed on the CHTC (Center for High Throughput Computing) cluster. We made use of CPU nodes with the following specifications:

- CPUs: 16 to 32 cores per job.
- Memory: Up to 128 GB.
- Disk space: Up to 60 GB

These resources were sufficient for running our model across multiple epochs and generating evaluation metrics on occupancy and flow predictions.

8 Limitations and Future Work

- **GPU Constraints.** Unfortunately, we were unable to leverage GPU acceleration during training due to containerization issues:
  - While we attempted to use TensorFlow Docker images that supported GPU, these images included a Python version that was incompatible with the Waymo Open Dataset API.
  - We explored options to build a custom Docker image with matching dependencies, but were constrained by time and cluster-specific environment configurations.
  - As a result, all experiments were CPU-bound, which limited both the speed of training and the extent to which we could explore larger models or longer sequence lengths.
- **Impact on Results.** The absence of GPU access and limited training time impacted the complexity of modeling we could afford:
  - Training on a larger dataset or running more exhaustive hyperparameter sweeps would have significantly benefited the model, especially for rare or occluded cases.
  - Incorporating more advanced temporal and contextual modeling—such as Transformers was not feasible within current compute limits.
  - Despite these limitations, the model achieved competitive performance on observed occupancy and reasonable flow predictions under constrained training conditions.

9 Collaboration plan

All team members contributed equally to the project. We collaborated through regular meetings, and joint working sessions to ensure cohesive development and shared responsibility.

10 References

[1] *Waymo Occupancy Flow Prediction Challenge (2024).* Retrieved from `https://waymo.com/open/challenges/2024/occupancy-flow-prediction/`

[2] *Waymo Open Dataset: Motion Dataset (2024).* Retrieved from `https://waymo.com/open/download/#motion-dataset`

[3] *Learning Traffic as Images* Retrieved from `https://pmc.ncbi.nlm.nih.gov/articles/PMC5422179/`

[4] *Spatio-Temporal vehicle traffic flow prediction* Retrieved from `https://www.sciencedirect.com/science/article/abs/pii/S2214785321031692`

[5] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". Available: `https://arxiv.org/abs/2010.11929`.

[6] X. Huang, X. Tian, J. Gu, Q. Sun, and H. Zhao,"VectorFlow: Combining Images and Vectors for Traffic Occupancy and Flow Prediction," arXiv, 2022. Available: `https://arxiv.org/abs/2208.04530`.

[7] Y. Murhij and D. Yudin, "OFMPNet: Deep End-to-End Model for Occupancy and Flow Prediction in Urban Environment," *arXiv*, 2024. Available: `https://arxiv.org/abs/2404.02263`.

[8] J. Ngiam, B. Caine, V. Vasudevan, Z. Zhang, H.-T. L. Chiang, J. Ling, R. Roelofs, A. Bewley, C. Liu, A. Venugopal, D. Weiss, B. Sapp, Z. Chen, and J. Shlens, "Scene Transformer: A Unified Multi-task Model for Behavior Prediction and Planning,", 2021. Available: `https://arxiv.org/abs/2106.08417`.